Parcial Práctico de Ingeniería de Software

La última transmisión — Motor de historias interactivas

Daniel Sanchez — Juan Diego Serpa

15 de octubre de 2026

Repositorio local ejecutable con `README.md`.

1. Problema, actores, alcance y calidad (U1)

1.1. Problema

Un colectivo de escritores prepara una ficción interactiva para un festival: una operadora recibe una señal de la estación Aurora, abandonada hace 20 años. Necesitan un **motor** que cargue narraciones desde archivos editables por no programadores, ofrezca solo opciones compatibles con lo ocurrido, aplique consecuencias y rechace contenido erróneo sin cierres inesperados. El equipo entrega motor + historia inicial de 6 escenas/3 finales + segunda historia + pruebas.

1.2. Actores y alcance

- **Lector:** lee escena actual, elige entre opciones habilitadas, reconoce el desenlace y reinicia sin arrastre de estado.
- **Escritor (no programa):** edita JSON con formato documentado; recibe diagnóstico preciso (causa + campo/ID) antes de jugar.
- **Coordinadora del festival:** exige que ningún error de contenido llegue a un cierre abrupto en vivo.

En alcance: carga JSON, evaluación de igualdades booleanas, efectos de asignación booleana, sesión en memoria, validación previa, CLI local, reinicio, bloqueo controlado. **Fuera de alcance:** cuentas, BD, guardado entre ejecuciones, red, editor visual, audio/imagen, lenguaje de expresiones (según límites del enunciado).

1.3. Riesgos (2)

1. **R1. Contenido inválido en vivo rompe la demo.** *Mitigación:* validación estricta previa en `validator.py` (9 categorías: ilegible, campos faltantes, inicio inexistente, IDs escena/opción duplicados, destino inexistente, variable no declarada, valor no booleano, final/no-final con número de opciones incompatible). Rechazo con mensaje + ID/campo, sin abrir sesión jugable; la CLI captura cualquier error de carga (JSON mal formado, codificación inválida, archivo inexistente) sin traza ni cierre abrupto.
2. **R2. Filtrado incorrecto de opciones en E3 (mostrar 1 o 3 en vez de 2).** *Mitigación:* evaluador central puro `evaluar_condiciones()` (conjunción de igualdades); prohibido hardcodear IDs de historia en el motor; pruebas CA01/CA02 y U1/U3 específicas.

1.4. Priorización de calidad

1. **Modificabilidad del contenido (1.º):** el cliente vive de agregar escenas sin programar. Se mide con RNF01 (segunda historia sin tocar código).
2. **Confiabilidad/robustez (2.º):** un fallo en festival es público. Se mide con RNF03 (100 % de la suite, 18 pruebas) y validación exhaustiva.
3. **Claridad de errores (3.º):** el escritor no programa. Se mide con RNF02 (mensaje con causa+ID en < 1 s).

Rendimiento y estética quedan subordinados: el motor es I/O mínimo y la consigna excluye valoración literaria/gráfica.

1.5. Responsabilidad profesional

Decisión: no inventar rutas ni reglas fuera del JSON entregado y no ocultar errores de contenido al lector con valores por defecto silenciosos. *Contexto:* sería fácil "autocompletar" una variable faltante como `false` para que la demo no caiga. *Consecuencia:* se rechaza el archivo con diagnóstico al escritor; se protege autoría del relato y se evita presentar al público una historia distinta a la escrita. Coherente

con ello, la CLI no sustituye un archivo inexistente por otro: informa el error. Dependencias limitadas a biblioteca estándar para no introducir riesgos de licencia/seguridad.

2. Proceso (U2)

Elegido: iterativo-incremental ligero (2 incrementos). Incremento A (horas 0–6): contrato JSON + validador + motor + CLI + pruebas U1–U5+I1. Punto de control hora 6: congelar versión inicial (tag `v0.1-pre-cambio`) aunque esté incompleta, leer solicitud de cambio y replanificar. Incremento B (horas 6–12): aplicar cambio con análisis de impacto y extender pruebas (mínimo 6 unitarios + 1 integración).

Alternativa descartada: cascada. Exigiría requisitos congelados, incompatible con el cambio obligatorio de mitad de ventana y con la validación temprana que piden N06/CA03.

Actividad	Cómo se realiza en esta entrega
Especificación	RF propios trazados a N01–N06 + 3 RNF medibles; 3 preguntas/supuestos; revisión cruzada con registro de defecto.
Desarrollo	Capas Contenido/Reglas/Interfaz; sin IDs de historia en código; convención: validador no juega, motor no lee ficheros.
Validación	`python -m unittest test_engine -v` (U1–U5, I1, categorías de error con `subTest`, bloqueo, CA01–CA03, RNF01–RNF02); recorridos manuales de 3 finales; segunda historia.
Evolución	Análisis de impacto (requisitos/modelos/componentes/pruebas), backlog ajustado, trazabilidad actualizada, tag `v1.0-post-cambio`.

3. Agilidad (U3)

3.1. Backlog (resumen)

BL1 Cargar+validar JSON (N01,N06). BL2 Filtrar opciones por estado (N02). BL3 Aplicar efectos y avanzar; entradas inválidas no mutan (N03,N04). BL4 Finales + reinicio limpio (N05). BL5 CLI legible + bloqueo controlado. BL6 Segunda historia sin tocar motor (RNF01). BL7 Suite U1–U5+I1 + 9 errores (RNF02,RNF03). BL8 Informe/RM/trazabilidad/video-guion. BL9 (hora 6) Cambio docente: renombrar `revisar_senal`→`revisar` en E1; actualizar pruebas/rutas; reverdecer suite.

Tablero Kanban (Por hacer / En curso / Hecho, con responsables I1/I2): captura inicial (hora ∼1, BL1–BL3 en curso) y captura final (hora ∼11, todo en Hecho salvo BL9 según cambio recibido) en Anexo C (`Anexo.pdf`). Historial Git con autores/fechas/mensajes se exporta con `git log --stat` para el ZIP.

3.2. Prácticas

P1. Definición de terminado (DoD): código + prueba automatizada en verde + README actualizado + trazabilidad. Evitó "casi listo" antes del corte de hora 6. **P2. Integración continua local:** correr la suite tras cada cambio de motor/validador; detectó el bug `json` no importado en `main.py`. **Descartada: pair-programming permanente.** Con solo 12 h asíncronas y dos personas, se sustituyó por revisión cruzada de RF y de PR (evidencia: defecto §4.3).

Aprendizaje + mejora: subestimamos la validación (9 categorías toman más que el motor). Acción: en BL9 reservar 30 min fijos a pruebas parametrizadas de errores antes de tocar funcionalidad nueva.

4. Requerimientos (U4)

Conflicto escritora vs. diseñador: ella pide retroceder siempre; él pide decisiones irreversibles. **Acuerdo provisional v1:** no implementar retroceso. **Pregunta de aclaración registrada:** *¿Debe existir "atrás" ilimitado, limitado a escena anterior, o ninguna?* Sin respuesta durante la ventana se mantiene el supuesto documentado y no se presupone decisión posterior.

4.1. RF propios (atómicos, con aceptación; cobertura N01–N06)

ID (N)	Comportamiento observable	Criterio de aceptación
RF01 (N01)	Cargar relato independiente: título, inicio, variables iniciales, escenas.	Dado `la_ultima_transmision.json`, carga sin errores; título/inicio/vars coinciden.
RF02 (N01)	Rechazar antes de jugar si hay error de contenido.	Dado destino E99 inexistente (CA03), se rechaza con mensaje que cita `E99`, sin sesión jugable.
RF03 (N02)	Mostrar solo opciones con todas sus condiciones cumplidas; mapa vacío = habilitada.	CA01: en E3 con `conoce_origen=true` se ven `activar_codigo`+ `cortar`, no `activar_sin_verificar`.
RF04 (N03)	Elección válida aplica efectos y mueve al destino.	CA01: `investigar` deja sesión en E3 con `conoce_origen=true`.
RF05 (N03)	Entrada inexistente o deshabilitada no muta estado.	CA02: enviar `activar_codigo` con `conoce_origen=false` lanza error y escena/vars intactos.
RF06 (N04)	La sesión recuerda vars durante la partida.	Secuencia E1→E2→E3 conserva `conoce_origen` hasta filtrar E3.
RF07 (N05)	Detectar final (cero opciones), anunciar desenlace y reiniciar limpio.	En E4/E5/E6 se anuncia FIN; tras reinicio escena=E1 y vars iniciales.

RF08 (N05+N02)	Si escena o final queda sin habilitadas, informar bloqueo y ofrecer reinicio.	Historia de prueba B2 sin habilitadas → mensaje de bloqueo + reinicio (prueba `test_bloqueo`).

Cobertura: N01→RF01,RF02; N02→RF03; N03→RF04,RF05; N04→RF06; N05→RF07,RF08; N06→RF02 (+RNF02). Cada RF es atómico y verificable por prueba nombrada.

4.2. RNF medibles

1. **RNF01 Modificabilidad:** cargar `segunda_historia.json` (≥3 escenas, 1 condición, 1 efecto) sin editar el motor. *Procedimiento:* `python main.py segunda_historia.json` + `test_rnf01_segunda_historia` y `test_rnf01_condicion_segunda_historia`. *Evidencia:* suite en verde (`resultados_pruebas.txt`); recorrido L1→L2→L4. *Umbral:* 0 líneas de motor modificadas.
2. **RNF02 Claridad de errores:** todo rechazo indica causa + campo/ID y tarda < 1 s. *Procedimiento:* `TestValidacion.test_u4_rechazo_con_causa_e_id` (19 variantes con `subTest`, cada una comprueba que el mensaje contenga el ID o campo) y `test_rnf02_rechazo_en_menos_de_un_segundo` (cronómetro `perf_counter`, umbral 1 s). *Evidencia:* mensaje real: "La opción 'revisar' en la escena 'E1' apunta a un destino inexistente 'E99'."; ambas pruebas en verde en `resultados_pruebas.txt`.
3. **RNF03 Confiabilidad:** 100 % de la suite en cada corrida local. *Procedimiento:* `python -m unittest test_engine -v` (18 pruebas). *Evidencia:* `Ran 18 tests ... OK` en `resultados_pruebas.txt`; se declara cualquier fallo pendiente (actualmente ninguno).

4.3. Preguntas/supuestos (3)

P1. Retroceso (arriba): supuesto = sin retroceso en v1. P2. Variables fuera del mapa inicial: supuesto = error de validación (no autocrear). P3. Entrada no numérica en CLI: supuesto = mensaje y reintento sin mutar estado (no cerrar programa, por N03/N06).

4.4. Revisión cruzada: defecto y corrección

Defecto: RF05 decía "ignorar opción inválida" pero no exigía estado intacto; U3 inicial solo comprobaba la excepción. **Corrección:** se reformuló RF05/U3 a "deja intacto el estado" y se agregó aserción escena+vars antes/después (pruebas `test_u3_*`). Detectado por Integrante 2 en revisión de RF vs. pruebas.

5. Modelado (U5)

Los 5 modelos (pequeños y legibles) están en Anexo A; aquí su lectura conjunta. **Contexto:** Lector/Escritor ↔ CLI ↔ Motor ↔ JSON. **Casos de uso:** Cargar historia, Jugar/ Elegir opción, Reiniciar, Diagnosticar error (incluye bloqueo). **Clases de dominio:** `Historia(titulo,inicio,variables,escenas)`, `Escena(id,texto,final,opciones)`, `Opción(id,texto,destino,condiciones,efectos)`, `Sesión(escenaActu` + `Motor` y `Validador` como servicios. **Secuencia Elegir:** CLI→Motor.opcionesDisponibles → Motor.aplicarEfectos → Motor.mover → CLI renderiza. **Estados de Sesión:** Inicio → EnPartida → {Final | Bloqueada} → Reiniciada. **Coherencia:** cada RF aparece en ≥1 caso de uso, ≥1 clase/método y ≥1 prueba (ver matriz §8); ningún método referencia IDs concretos de historia (E1/E2...), garantizando RNF01.

6. Arquitectura (U6)

Tres capas con dependencias hacia adentro: **Interfaz (`main.py`) → Reglas (`engine.py`) → Contenido (JSON + `validator.py`)**. Responsabilidades: Interfaz solo presenta/pide; Motor evalúa/aplica/mueve; Validador rechaza antes de jugar. Diagrama en Anexo B.

D1. Validación separada y previa vs. validar al vuelo. *Alternativas:* (a) validar todo antes de crear sesión; (b) validar cada acceso. *Elección:* (a). *Consecuencias +:* CA03 simple, sin sesiones a medio cargar; mensajes completos. *–:* no soporta edición en caliente (fuera de alcance).

D2. Motor genérico por datos vs. ramificado por historia. *Alternativas:* (a) evaluar condiciones desde mapas; (b) `if escena==E3...`. *Elección:* (a). *Consecuencias +:* RNF01 demostrable (segunda historia con IDs L1…funciona); E3 muestra dinámicamente 2/3 opciones. *–:* expresividad limitada a igualdades booleanas (límite aceptado del ejercicio).

Separación demostrada: `segunda_historia.json` (Laboratorio, L1–L4, condición `tiene_llave=true`, efecto `tiene_llave:=true`) se ejecuta con el mismo `main.py/engine.py` sin cambios (prueba + recorrido manual en §7.1).

7. Implementación y pruebas (U7)

Diseño detallado: `StoryEngine` concentra sesión (escenaActual+vars) y reglas; `obtener_opciones_disponib` es consulta pura; `tomar_opcion()` valida contra las disponibles y solo entonces muta (efectos y luego destino), lo que hace U3 trivialmente cierto. Acoplamiento bajo: Motor no conoce CLI ni ficheros (recibe dict ya validado); Validador no conoce sesión.

Bibliotecas y licencias: solo biblioteca estándar Python 3 (`json, unittest, os, sys, copy, tempfile, time`) — PSF License, sin dependencias externas; instalación cero. Versión probada: Python 3.14.4.

7.1. Pruebas automatizadas

`python -m unittest test_engine -v` — **18 pruebas, OK** (registro: `resultados_pruebas.txt`). **U1:** `test_u1_*` (condición cumplida y no cumplida en E3, mapa vacío habilita). **U2:** `test_u2_*` (efectos y destino E1→E4; efecto falso en `cortar`). **U3:** `test_u3_*` (opción inexistente, vacía, ID antiguo `revisar_senal` y deshabilitada CA02 dejan escena y variables intactas). **U4:** `TestValidacion` (19 variantes de error con `subTest`; archivo ilegible: JSON mal formado, no UTF-8 y vacío; archivo inexistente; CA03). **U5:** `test_u5_reiniciar` restaura escena y ambas variables tras un final. **I1:** `test_i1_tres_finales` carga el archivo real y alcanza E4, E5 y E6. Además: bloqueo con reinicio (`test_bloqueo_*`) y segunda historia (RNF01).

7.2. Verificación del producto y RNF

Tres finales recorridos en CLI (rutas en `README.md` §Rutas). Segunda historia L1→L2→L4 en verde. RNF01: 0 cambios de motor. RNF02: mensajes con causa+ID, cada rechazo tarda < 1 s (verificado por prueba automatizada). RNF03: 18/18 en verde en última corrida; sin fallos pendientes en flujo base. Capturas/manual complementan pero no sustituyen la suite.

8. Cambio y trazabilidad

Versión de partida (hora 6): tag `v0.1-pre-cambio` = motor+validador+CLI+suite previa en verde (7/7) + historia con opción `revisar_senal` en E1. **Solicitud de cambio recibida (hora 6, archivo docente `la_ultima_transmision.json`):** renombrar en E1 el ID de opción `revisar_senal` → `revisar` (texto "Revisar señal", destino E2, sin condiciones/efectos; resto de escenas, variables y

reglas intactas). **Análisis de impacto:** cambio solo de datos, sin alterar contrato JSON; requisitos RF01–RF08 y modelos del Anexo intactos; componentes afectados: `test_engine.py` (3 llamadas) y rutas del `README.md`; motor/validador sin cambios, lo que confirma la separación contenido-reglas (RNF01). **Pruebas:** suite actualizada a `tomar_opcion(‘‘revisar’’)` en U1/U3/U5; el **escenario unitario nuevo** es el rechazo del ID antiguo `revisar_senal` como opción inexistente con estado intacto (`test_u3_opcion_inexistente`, subcaso `revisar_senal`); U4/CA03 y bloqueo sin cambios de comportamiento. **Evidencia:** diff `v0.1-pre-cambio..v1.0-post-cambio` + `python -m unittest test_engine -v` en verde (18/18) + matriz §8 actualizada.

RF/RNF	Modelo (Anexo)	Código/archivo	Evidencia
RF01,RF02	CU Cargar; Clases Historia/Escena	`validator.py`, `engine.py:__init__`, JSONs	`TestValidacion.test_u4_*`, `test_ca03_*`
RF03	CU Elegir; Secuencia; Estados	`engine.py:evaluar_condiciones`, `obtener_opciones_disponibles`	`test_u1_*`
RF04–RF06	Secuencia; Clases Sesión/Opción	`engine.py:tomar_opcion`	`test_u2_*`, `test_u3_*`
RF07,RF08	Estados; CU Reiniciar	`engine.py:reiniciar`, `main.py:jugar`	`test_u5_reiniciar`, `test_i1_tres_finales`, `test_bloqueo_*`
RNF01	Componentes (separación)	`segunda_historia.json`	`test_rnf01_*`
RNF02	—	`validator.py` (mensajes)	`test_u4_rechazo_con_causa_e_id`, `test_rnf02_*`
RNF03	—	`test_engine.py`	`resultados_pruebas.txt: Ran 18 tests OK`

9. Aportes, fuentes y límites

Aportes: DANIEL SANCHEZ: motor/CLI/historias JSON. JUAN SERPA: validador/suite RF-RNF/informe. Ambos: revisión cruzada, recorridos finales y video-guion. Detalle por archivo/tarea en historial Git exportado del ZIP. **Fuentes:** cada resultado se verificó ejecutando la suite (`resultados_pruebas.txt`) y recorriendo los tres finales en la CLI; sin fragmentos de terceros con licencia restrictiva. **Límites conocidos:** sin persistencia/red/cuentas; sin retroceso (acuerdo provisional); solo booleanos; sin análisis exhaustivo de caminos; video de demostración incluido en el ZIP o enlazado desde el `README.md`.